

SVM

Rodrigo Mansilla, Javier Chen

2025-02-25

Exploración de Datos

Table 1: Número de valores NA por columna

Columna	Cantidad de NA
MasVnrType	4
MasVnrArea	4
BsmtQual	26
BsmtCond	26
GarageYrBlt	57
GarageQual	57
GarageCond	57

Table 2: Umbrales para categorizar PriceCat

Cuartil	SalePrice
25% (Eco/Inter)	-1
75% (Inter/Caras)	1

Table 3: Distribución de categorías de precio

PriceCat	Train	Test
Economicas	256	109
Intermedias	509	219
Caras	259	108

Table 4: Dimensiones de los datasets con PriceCat

Dataset	Filas	Columnas
train_cat	1024	104
test_cat	436	104

- Focos de datos faltantes: La mayoría de los NA se concentran en atributos de sótano y garaje (**BsmtQual**, **BsmtCond**, **GarageYrBlt**, **GarageQual**, **GarageCond**), lo que sugiere revisar si imputarlos con la mediana o tratarlos como “sin sótano/garaje” para no perder información clave.

- Umbrales de PriceCat: Se fijaron en los cuartiles 25 % (`SalePrice < -1`) y 75 % (`SalePrice > 1`), garantizando cortes claros entre “Económicas”, “Intermedias” y “Caras”.
 - Balance de clases:
 - "Intermedias" es la más numerosa (~50 % del total).
 - "Económicas" y "Caras" están pareadas en tamaño (~25 % cada una), aunque "Intermedias" podría dominar.
 - Consistencia train/test: La proporción de cada categoría en entrenamiento (256/509/259) y prueba (109/219/108) preserva la distribución original, favoreciendo evaluaciones comparables.
 - Volumen de datos:
 - Train_cat: 1 024 filas × 104 columnas
 - Test_cat: 436 filas × 104 columnas
- Suficiente para entrenar modelos SVM sin riesgo excesivo de varianza por escasez de datos.
- Implicación para modelado: Dado el tamaño y la ligera clase mayoritaria en “Intermedias”, conviene monitorizar métricas más allá del accuracy (e.g. F1-score por clase) y considerar estratificación o ponderación en el entrenamiento.

Entrenamiento del modelo SVM

Table 5: Comparación de SVM: Train vs Test Accuracy

Modelo	Kernel	Grado	Costo	Gamma	Acc. Train	Acc. Test
svm_lin_C0.1	linear	NA	0.1	NA	0.958	0.812
svm_lin_C1	linear	NA	1.0	NA	0.982	0.803
svm_lin_C10	linear	NA	10.0	NA	0.991	0.800
svm_poly_deg2_C1_g0.01	polynomial	2	1.0	0.010	0.973	0.805
svm_poly_deg3_C1_g0.01	polynomial	3	1.0	0.010	0.968	0.791
svm_poly_deg2_C1_g0.1	polynomial	2	1.0	0.100	1.000	0.787
svm_poly_deg3_C1_g0.1	polynomial	3	1.0	0.100	1.000	0.775
svm_poly_deg2_C5_g0.01	polynomial	2	5.0	0.010	0.995	0.800
svm_poly_deg3_C5_g0.01	polynomial	3	5.0	0.010	0.997	0.784
svm_poly_deg2_C5_g0.1	polynomial	2	5.0	0.100	1.000	0.787
svm_poly_deg3_C5_g0.1	polynomial	3	5.0	0.100	1.000	0.775
svm_rad_C1_g0.001	radial	NA	1.0	0.001	0.887	0.812
svm_rad_C1_g0.01	radial	NA	1.0	0.010	0.973	0.814
svm_rad_C1_g0.1	radial	NA	1.0	0.100	1.000	0.507
svm_rad_C10_g0.001	radial	NA	10.0	0.001	0.951	0.853
svm_rad_C10_g0.01	radial	NA	10.0	0.010	1.000	0.789
svm_rad_C10_g0.1	radial	NA	10.0	0.100	1.000	0.509

Table 6: Resultados de tuning kernel radial

Gamma	Cost	Error	SD
0.125	0.5	0.498	0.069
0.250	0.5	0.498	0.069
0.500	0.5	0.498	0.069
0.125	1.0	0.497	0.069
0.250	1.0	0.498	0.069
0.500	1.0	0.498	0.069
0.125	2.0	0.493	0.066
0.250	2.0	0.498	0.069
0.500	2.0	0.498	0.069
0.125	4.0	0.493	0.066
0.250	4.0	0.498	0.069
0.500	4.0	0.498	0.069

Table 7: Mejor modelo radial

Mejor_Kernel	Costo	Gamma
radial	2	0.125

- Lineal vs. polinomial vs. radial
 - Lineal ($C=0.1-10$): buena precisión de prueba ($\sim 0.80-0.81$) con brecha Train-Test de $\sim 14-19$ puntos, sugiere algo de sobreajuste al subir C .
 - Polinomial (grado 2-3): al aumentar C o alcanza 100 % en entrenamiento pero cae a <0.79 en prueba, señal clara de overfitting.
 - Radial ($C=1-10$, $\gamma=0.001-0.1$): el mejor balance lo da $C=10$, $\gamma=0.001$ (Train 95.1 %, Test 85.3 %, Δ 9.8 pp), reduciendo el sobreajuste respecto a lineal/polinomial.
- Efecto de los hiperparámetros (Table 6-7)
- Grid search CV identificó como óptimos para kernel radial $C=2$ y $\gamma=0.125$, con error medio 0.493 (SD 0.069)
- Brecha Train-Test
 - La menor diferencia (~ 9.8 pp) se ve en radial $C=10$, $\gamma=0.001$; lineal y polinomial muestran diferencias de ~ 22 pp), confirmando que el kernel radial generaliza mejor.
- Selección del mejor modelo
 - Basado en Test Accuracy y brecha moderada, el SVM radial es el ganador, y el ajuste fino ($C=2$, $\gamma=0.125$) es el mejor.

Predicciones

Table 8: Accuracy de SVM en test_final

Modelo	Accuracy
--------	----------

svm_rad_C10_g0.001	0.8532110
svm_rad_C1_g0.01	0.8142202
svm_lin_C0.1	0.8119266
svm_rad_C1_g0.001	0.8119266
svm_poly_deg2_C1_g0.01	0.8050459
svm_lin_C1	0.8027523
svm_lin_C10	0.8004587
svm_poly_deg2_C5_g0.01	0.8004587
svm_poly_deg3_C1_g0.01	0.7912844
svm_rad_C10_g0.01	0.7889908
svm_poly_deg2_C1_g0.1	0.7866972
svm_poly_deg2_C5_g0.1	0.7866972
svm_poly_deg3_C5_g0.01	0.7844037
svm_poly_deg3_C1_g0.1	0.7752294
svm_poly_deg3_C5_g0.1	0.7752294
svm_rad_C10_g0.1	0.5091743
svm_rad_C1_g0.1	0.5068807
svm_best_radial	0.5045872

- svm_lin_Cx
 - lin: kernel lineal.
 - C=x: parámetro de penalización (regularización) igual a x.
- svm_poly_degD_Cx_gY
 - poly: kernel polinomial.
 - degD: grado del polinomio D.
 - C=x: coste de error x.
 - g=Y: gamma (influye en el "alcance" de cada punto de soporte) igual a Y.
- svm_rad_Cx_gY
 - rad: kernel radial (RBF).
 - C=x: coste de penalización x.
 - g=Y: gamma de la función RBF igual a Y.
- svm_best_radial
 - Modelo radial ajustado automáticamente con los hiperparámetros que dieron menor error en validación.

Insights

- El mejor modelo es svm_rad_C10_g0.001 con 85.32 % de accuracy en test, claramente por encima del resto.

- Le siguen de cerca svm_rad_C1_g0.01 (81.42 %) y svm_lin_C0.1 (81.19 %), lo que muestra que un kernel lineal bien regularizado compite con versiones radiales de bajo γ .
- Los polinomiales alcanzan como máximo ~80.5 % (grado 2, $C=1$, $\gamma=0.01$), indicando que su complejidad extra no se traduce en mejor performance.
- Las configuraciones con $\gamma=0.1$ (tanto lineal como radial) colapsan a ~50 %, un claro síntoma de overfitting que las hace inútiles para clasificación de tres clases.
- La coherencia entre los resultados de CV (Table 6–7) y estas accuracies confirma que los mejores rangos de γ están en $[0.001–0.01]$.
- Recomendación: quedarse con svm_rad_C10_g0.001 y, de ser posible, probar la combinación CV-óptima ($C=2$, $\gamma=0.125$) para ver si mejora aún más la generalización.

Matriz de confusión

Table 9: Matriz de confusión — svm_lin_C0.1

Referencia	Economicas	Intermedias	Caras
Economicas	86	23	0
Intermedias	17	183	19
Caras	0	23	85

Table 10: Matriz de confusión — svm_lin_C1

Referencia	Economicas	Intermedias	Caras
Economicas	84	25	0
Intermedias	18	180	21
Caras	0	22	86

Table 11: Matriz de confusión — svm_lin_C10

Referencia	Economicas	Intermedias	Caras
Economicas	83	26	0
Intermedias	21	175	23
Caras	0	17	91

Table 12: Matriz de confusión — svm_rad_C1_g0.001

Referencia	Economicas	Intermedias	Caras
Economicas	77	32	0
Intermedias	14	199	6
Caras	0	30	78

Table 13: Matriz de confusión — svm_rad_C1_g0.01

Referencia	Economicas	Intermedias	Caras
Economicas	80	29	0
Intermedias	14	199	6
Caras	0	32	76

Table 14: Matriz de confusión — svm_rad_C1_g0.1

Referencia	Economicas	Intermedias	Caras
Economicas	0	109	0
Intermedias	0	219	0
Caras	0	106	2

Table 15: Matriz de confusión — svm_rad_C10_g0.001

Referencia	Economicas	Intermedias	Caras
Economicas	88	21	0
Intermedias	16	195	8
Caras	0	19	89

Table 16: Matriz de confusión — svm_rad_C10_g0.01

Referencia	Economicas	Intermedias	Caras
Economicas	83	26	0
Intermedias	21	183	15
Caras	0	30	78

Table 17: Matriz de confusión — svm_rad_C10_g0.1

Referencia	Economicas	Intermedias	Caras
Economicas	1	108	0
Intermedias	0	218	1
Caras	0	105	3

Table 18: Matriz de confusión — svm_poly_deg2_C1_g0.01

Referencia	Economicas	Intermedias	Caras
Economicas	76	29	4
Intermedias	12	198	9
Caras	0	31	77

Table 19: Matriz de confusión — svm_poly_deg2_C1_g0.1

Referencia	Economicas	Intermedias	Caras
Economicas	79	26	4
Intermedias	17	185	17
Caras	0	29	79

Table 20: Matriz de confusión — svm_poly_deg2_C5_g0.01

Referencia	Economicas	Intermedias	Caras
Economicas	80	25	4
Intermedias	18	191	10
Caras	0	30	78

Table 21: Matriz de confusión — svm_poly_deg2_C5_g0.1

Referencia	Economicas	Intermedias	Caras
Economicas	79	26	4
Intermedias	17	185	17
Caras	0	29	79

Table 22: Matriz de confusión — svm_poly_deg3_C1_g0.01

Referencia	Economicas	Intermedias	Caras
Economicas	67	42	0
Intermedias	10	204	5
Caras	0	34	74

Table 23: Matriz de confusión — svm_poly_deg3_C1_g0.1

Referencia	Economicas	Intermedias	Caras
Economicas	76	33	0
Intermedias	19	184	16
Caras	0	30	78

Table 24: Matriz de confusión — svm_poly_deg3_C5_g0.01

Referencia	Economicas	Intermedias	Caras
Economicas	73	36	0
Intermedias	18	191	10
Caras	0	30	78

Table 25: Matriz de confusión — svm_poly_deg3_C5_g0.1

Referencia	Economicas	Intermedias	Caras
Economicas	76	33	0
Intermedias	19	184	16
Caras	0	30	78

Table 26: Matriz de confusión — svm_best_radial

Referencia	Economicas	Intermedias	Caras
Economicas	0	109	0
Intermedias	0	218	1
Caras	0	106	2

- **Confusiones entre clases vecinas**

Casi todos los errores ocurren intercambiando solo con la clase adyacente (económicas → intermedias, intermedias → caras), lo cual es deseable porque evita clasificaciones “extremas” (económicas → caras).

- **Mejor desempeño global**

- ``svm_rad_C10_g0.001`` registra las tasas más bajas de confusión:

- Económicas→Intermedias: 21
- Intermedias→Económicas: 16
- Caras→Intermedias: 19

- **Modelos lineales bien balanceados**

- Con $C=0.1$, los errores son 23 econ→int, 17 int→econ, 19 int→caras, 23 caras→int.
- Aumentar C (hasta 10) reduce errores inter→caras (de 19→23 a 23→23) pero aumenta ligeramente econ→int.

- **Kernel polinomial grado 2**

- Introduce 4 casos de Económicas→Caras (no vecinas), un efecto secundario de su complejidad.
- Por lo demás, sus patrones de confusión son comparables a los lineales.

- **Radial con γ alto (0.1)**

- Modelos ``svm_rad_C1_g0.1`` y ``svm_rad_C10_g0.1`` colapsan prediciendo casi todo como "Intermedias" (y algunas veces "Caras").

- **Importancia de γ y C**

- γ muy pequeño (0.001-0.01) y C moderado-alto (1-10) equilibran sensibilidad y especificidad por clase.
- γ elevado lleva a decisiones extremas; C demasiado alto en polinomial también provoca 100 % train vs test.

- **Recomendación práctica**

- Mantener ``svm_rad_C10_g0.001`` y explorar el ajuste fino sugerido por CV (p.ej. $C=2$, $\gamma=0.125$) para γ y C .

Sobreajuste y validación cruzada

Table 27: Accuracy de entrenamiento vs. prueba para todos los modelos SVM

Modelo	Accuracy (Train)	Accuracy (Test)	Diferencia
svm_lin_C0.1	95.8%	81.2%	14.6 p.p.
svm_lin_C1	98.2%	80.3%	18.0 p.p.
svm_lin_C10	99.1%	80.0%	19.1 p.p.
svm_rad_C1_g0.001	88.7%	81.2%	7.5 p.p.
svm_rad_C1_g0.01	97.3%	81.4%	15.8 p.p.
svm_rad_C1_g0.1	100.0%	50.7%	49.3 p.p.
svm_rad_C10_g0.001	95.1%	85.3%	9.8 p.p.
svm_rad_C10_g0.01	100.0%	78.9%	21.1 p.p.
svm_rad_C10_g0.1	100.0%	50.9%	49.1 p.p.
svm_poly_deg2_C1_g0.01	97.3%	80.5%	16.8 p.p.
svm_poly_deg2_C1_g0.1	100.0%	78.7%	21.3 p.p.
svm_poly_deg2_C5_g0.01	99.5%	80.0%	19.5 p.p.
svm_poly_deg2_C5_g0.1	100.0%	78.7%	21.3 p.p.
svm_poly_deg3_C1_g0.01	96.8%	79.1%	17.6 p.p.
svm_poly_deg3_C1_g0.1	100.0%	77.5%	22.5 p.p.
svm_poly_deg3_C5_g0.01	99.7%	78.4%	21.3 p.p.
svm_poly_deg3_C5_g0.1	100.0%	77.5%	22.5 p.p.
best_radial	100.0%	50.5%	49.5 p.p.

Análisis de sobreajuste (overfitting) y subajuste (underfitting)

- **Overfitting extremo:** modelos con **Accuracy(Train)=100 %** pero **Accuracy(Test) 80 %** (p.ej. `svm_rad_C1_g0.1`, `svm_rad_C10_g0.1`, `best_radial`) muestran brechas de ~49 pp. Están memorizando el training set y no generalizan.
- **Overfitting moderado:** modelos lineales con $C = 1$ (`svm_lin_C1`, `svm_lin_C10`) y polinomiales con 0.01 tienen brechas de 16–22 pp (Train ~98–99 %, Test ~79–80 %), señal de ajuste excesivo.
- **Buen balance:** `svm_rad_C1_g0.001` ($\Delta=7.5$ pp) y `svm_rad_C10_g0.001` ($\Delta=9.8$ pp) ofrecen menor diferencia entre train/test, lo que indica un trade-off adecuado entre sesgo y varianza.
- **Subajuste leve:** `svm_rad_C1_g0.001` (Train 88.7 %, Test 81.2 %) podría no capturar toda la complejidad, pero su baja brecha lo hace robusto frente al overfitting.

Estrategias para manejar over-/under-fitting

- **Validación cruzada estratificada (k-fold CV)**
 - Usar k-fold con estratificación por `PriceCat` para estimar más confiablemente la performance y evitar elecciones de hiperparámetros basadas en un único split.
- **Ajuste de hiperparámetros**
 - **Reducir** en kernels RBF/polinomiales para suavizar la frontera.
 - **Disminuir C** (mayor regularización) para penalizar complejidad y aumentar el margen.

- **Selección y reducción de variables**
 - Aplicar **PCA** o eliminar features de baja importancia (p.ej. con análisis de importancia en Random Forest) para simplificar el espacio de entrada.
- **Ensamble de modelos**
 - Combinar varios SVM con distintos kernels o algoritmos base (voting, bagging) para mitigar la varianza.